

Custom Chatbot

```
In [9]: import warnings
warnings.filterwarnings('ignore')
```

```
In [10]: import json
import numpy as np
import pandas as pd
import random
from matplotlib import pyplot as plt
import seaborn as sns
from wordcloud import WordCloud, STOPWORDS
import missingno as msno

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

import keras
from keras.models import Sequential
from keras.layers import Dense, Embedding, LSTM, Dropout
from keras.callbacks import ReduceLROnPlateau

from tensorflow.keras.preprocessing.sequence import pad_sequences
import nltk
from nltk import word_tokenize
from nltk.stem import PorterStemmer

import torch
from torch.utils.data import Dataset

from transformers import AutoTokenizer, TFAutoModelForSequenceClassification
from transformers import pipeline
from transformers import DistilBertTokenizerFast
from transformers import BertForSequenceClassification, BertTokenizerFast
from transformers import TFDistilBertForSequenceClassification, TFTrainingArguments
from transformers import BertTokenizer, TFBertForSequenceClassification, BertConfig
from transformers import TrainingArguments, Trainer

nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\BaddD\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
Out[10]: True
```

```
In [11]: def load_json_file(filename):
    with open(filename) as f:
        file = json.load(f)
    return file

filename = 'dataset/intents.json'
```

```
intents = load_json_file(filename)
```

```
In [12]: def create_df():
          df = pd.DataFrame({
              'Pattern' : [],
              'Tag' : []
          })

          return df

df = create_df()
df
```

```
Out[12]:
```

	Pattern	Tag
--	---------	-----

```
In [13]: def extract_json_info(json_file, df):

          for intent in json_file['intents']:

              for pattern in intent['patterns']:

                  sentence_tag = [pattern, intent['tag']]
                  df.loc[len(df.index)] = sentence_tag

          return df

df = extract_json_info(intents, df)
df.head()
```

```
Out[13]:
```

	Pattern	Tag
0	Hi	greeting
1	How are you?	greeting
2	Is anyone there?	greeting
3	Hello	greeting
4	Good day	greeting

```
In [14]: df2 = df.copy()
df2.head()
```

Out[14]:

	Pattern	Tag
0	Hi	greeting
1	How are you?	greeting
2	Is anyone there?	greeting
3	Hello	greeting
4	Good day	greeting

```
In [15]: def print_shape_df(df, ds_name="df"):
          print(f"{ds_name} dataset has {df.shape[0]} rows and {df.shape[1]} columns")

          print_shape_df(df, "Chatbot")
```

Chatbot dataset has 405 rows and 2 columns

```
In [16]: def print_dfInfo(df, ds_name="df"):
          print(f"The info of {ds_name} dataset\n")
          print(df.info())

          print_dfInfo(df, "Chatbot")
```

The info of Chatbot dataset

```
<class 'pandas.core.frame.DataFrame'>
Index: 405 entries, 0 to 404
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Pattern  405 non-null      object
1   Tag      405 non-null      object
dtypes: object(2)
memory usage: 9.5+ KB
None
```

```
In [17]: def num_classes(df, target_col, ds_name="df"):
          print(f"The {ds_name} dataset has {len(df[target_col].unique())} classes")

          num_classes(df, 'Tag', "Chatbot")
```

The Chatbot dataset has 38 classes

```
In [18]: def check_null(df, ds_name='df'):
          print(f"Null Values in each col in the {ds_name} dataset:\n")
          print(df.isnull().sum())

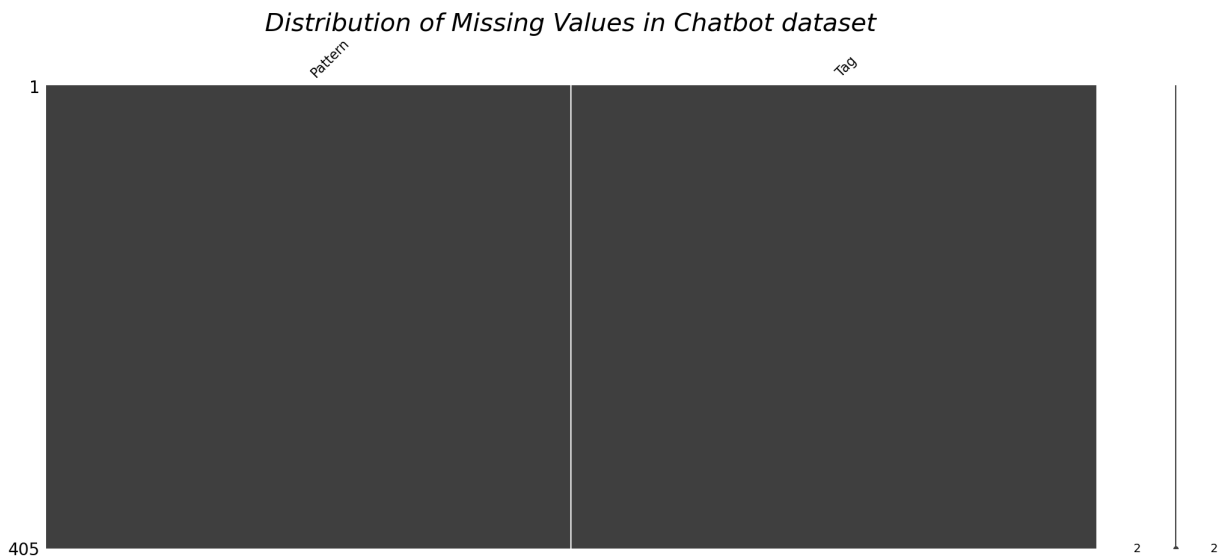
          check_null(df, "Chatbot")
```

Null Values in each col in the Chatbot dataset:

```
Pattern    0
Tag        0
dtype: int64
```

```
In [19]: def plot_miss_vals(df, ds_name="df"):
    msno.matrix(df)
    plt.title(f'Distribution of Missing Values in {ds_name} dataset', fontsize=30,
    plt.show()

    plot_miss_vals(df, "Chatbot")
```

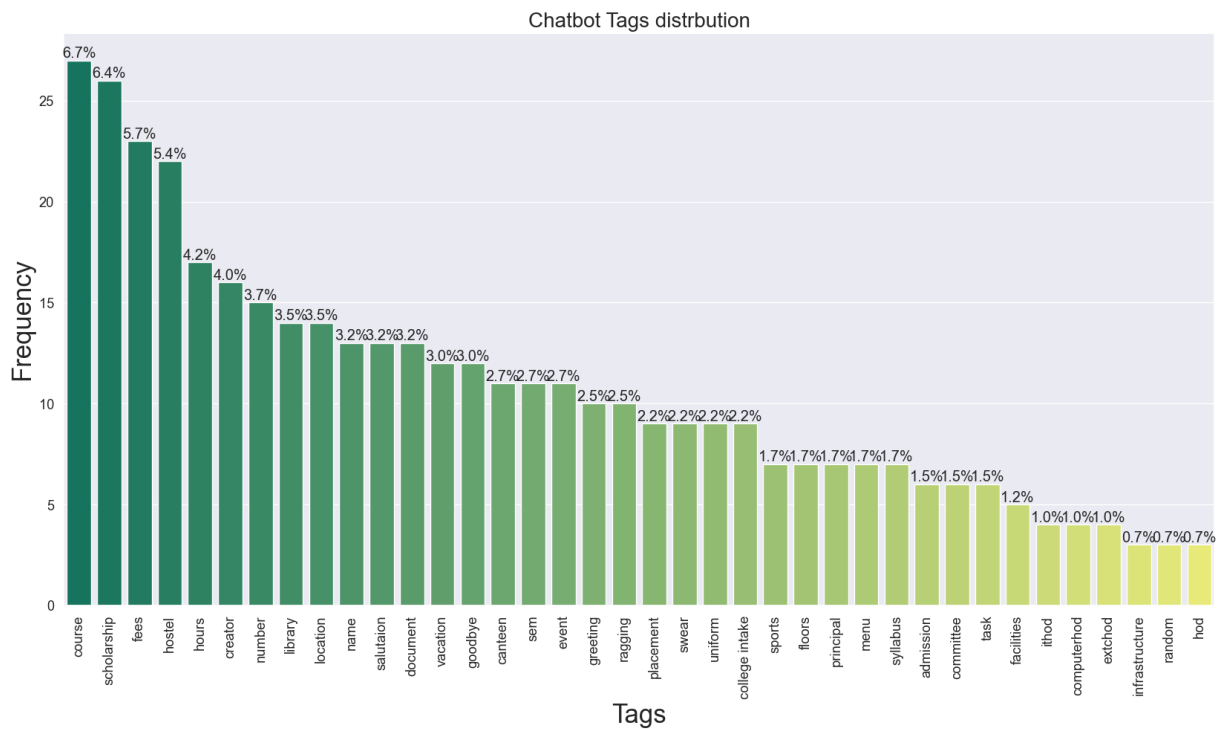


```
In [20]: def count_plot(x, df, title, xlabel, ylabel, width, height, order = None, rotation=
    ncount = len(df)
    plt.figure(figsize=(width,height))
    ax = sns.countplot(x = x, palette=palette, order = order, hue=hue)
    plt.title(title, fontsize=20)
    if rotation:
        plt.xticks(rotation = 'vertical')
    plt.xlabel(xlabel, fontsize=25)
    plt.ylabel(ylabel, fontsize=25)

    ax.yaxis.set_label_position('left')
    for p in ax.patches:
        x=p.get_bbox().get_points()[:,0]
        y=p.get_bbox().get_points()[1,1]
        ax.annotate('{:.1f}%'.format(100.*y/ncount), (x.mean(), y),
                    ha='center', va='bottom') # set the alignment of the text

    plt.show()
```

```
In [21]: sns.set(font_scale = 1.2)
    order = df['Tag'].value_counts().index
    count_plot(df['Tag'], df, "Chatbot Tags distrbution", "Tags", "Frequency", 20,10, o
```



```
In [22]: stemmer = PorterStemmer()
ignore_words=['?', '!', ',', '.']

def preprocess_pattern(pattern):
    words = word_tokenize(pattern.lower())
    stemmed_words = [stemmer.stem(word) for word in words if word not in ignore_words]
    return " ".join(stemmed_words)

df['Pattern'] = df['Pattern'].apply(preprocess_pattern)
df.head()
```

```
Out[22]:
```

	Pattern	Tag
0	hi	greeting
1	how are you	greeting
2	is anyon there	greeting
3	hello	greeting
4	good day	greeting

```
In [23]: print(df['Pattern'].isnull().sum())

0
```

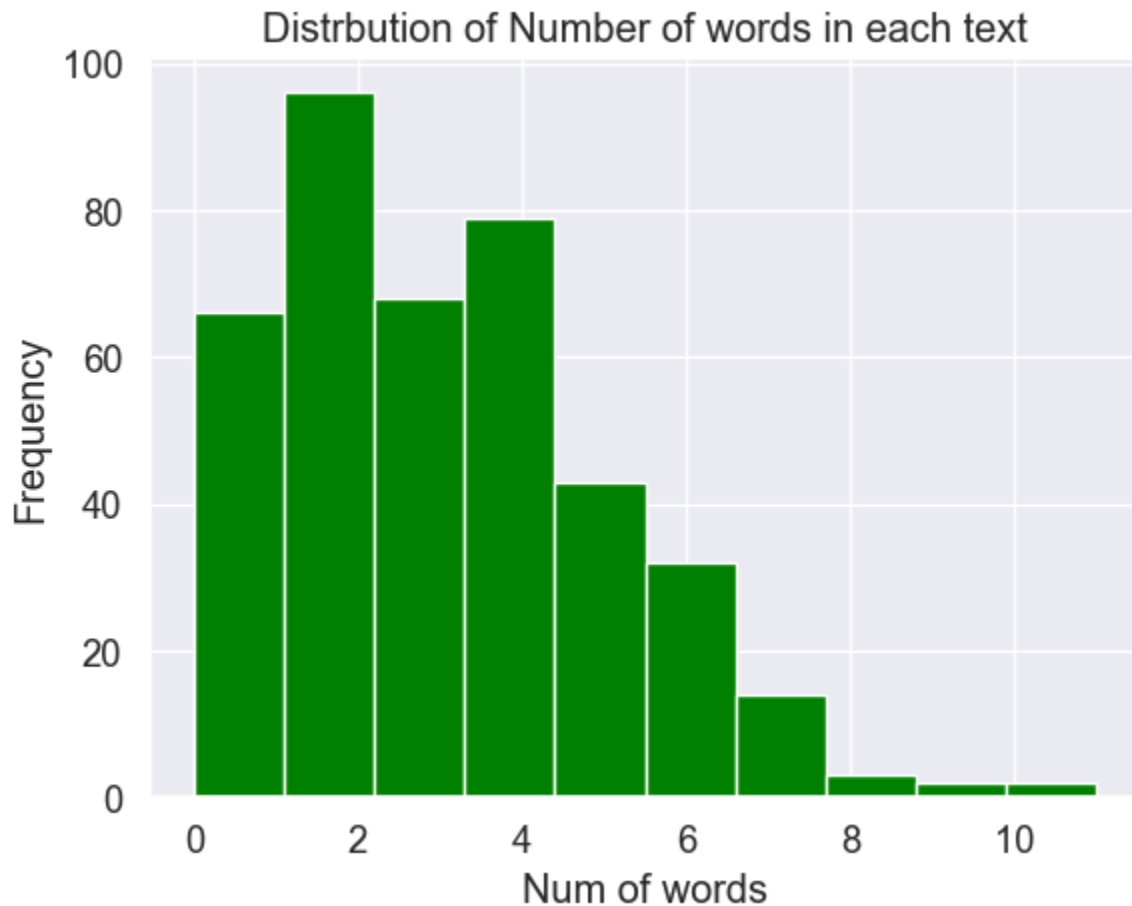
```
In [24]: plt.figure(figsize=(20, 20))
wc = WordCloud(max_words=2000, width=1600, height=800, stopwords=STOPWORDS).generate_from_texts(df['Pattern'].tolist())
plt.imshow(wc, interpolation='bilinear')
plt.xticks([])
plt.yticks([])
plt.show()
```

Distribution of Number of letters in each text

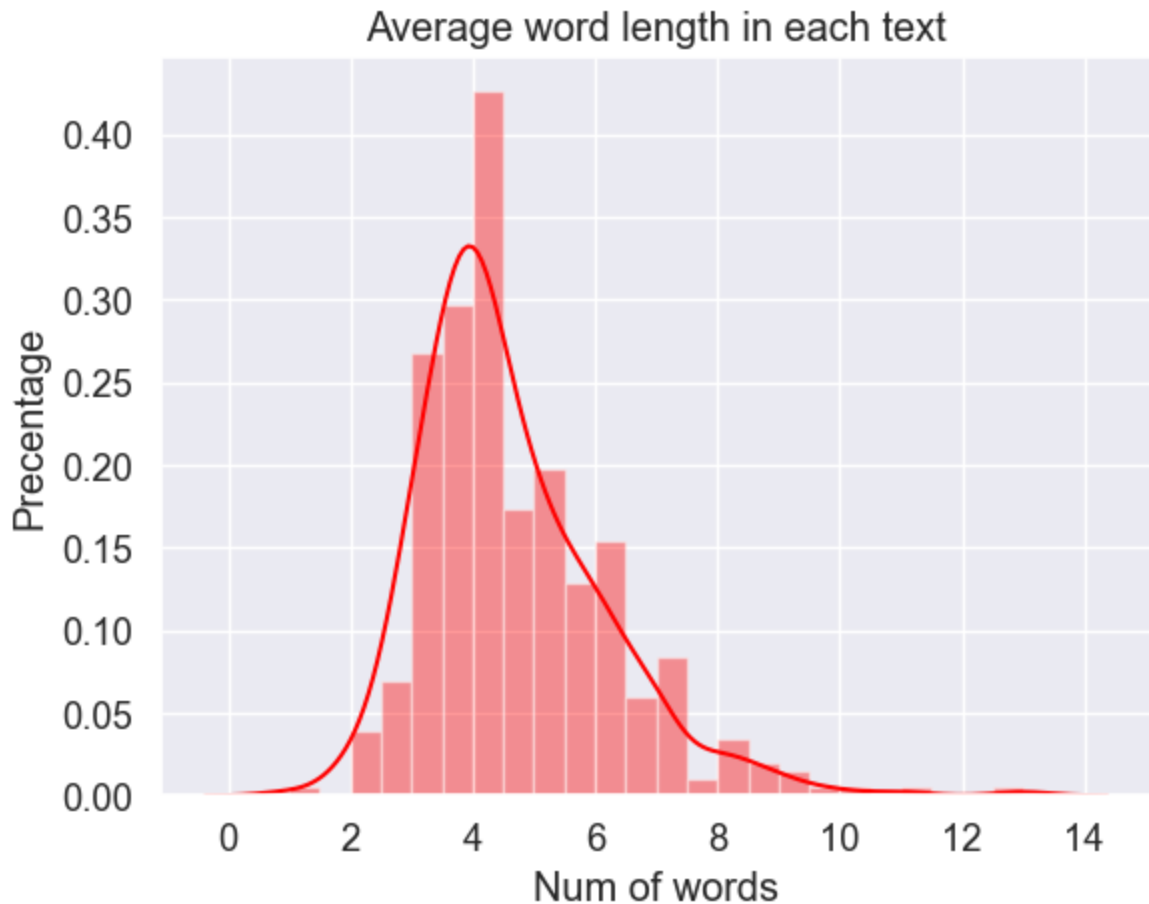
Num of letters	Frequency
0-5	25
5-10	62
10-15	98
15-20	69
20-25	73
25-30	36
30-35	21
35-40	15
40-45	5
45-50	2

08/01/2026, 15:24

```
plt.hist(words_len,color='green')
plt.title("Distribution of Number of words in each text")
plt.xlabel("Num of words")
plt.ylabel("Frequency")
plt.show()
```



```
In [27]: word=df['Pattern'].str.split().apply(lambda x : [len(i) for i in x])
word = word.map(lambda x: np.mean(x))
sns.distplot(word,color='red')
plt.title("Average word length in each text")
plt.xlabel("Num of words")
plt.ylabel("Precentage")
plt.show()
```



```
In [28]: def get_corpus(series):  
        words = []  
        for text in series:  
            for word in text.split():  
                words.append(word.strip())  
        return words  
  
        corpus = get_corpus(df.Pattern)  
        corpus[:5]
```

```
Out[28]: ['hi', 'how', 'are', 'you', 'is']
```

```
In [29]: print(f"dataset contains {len(corpus)} words")  
  
dataset contains 1345 words
```

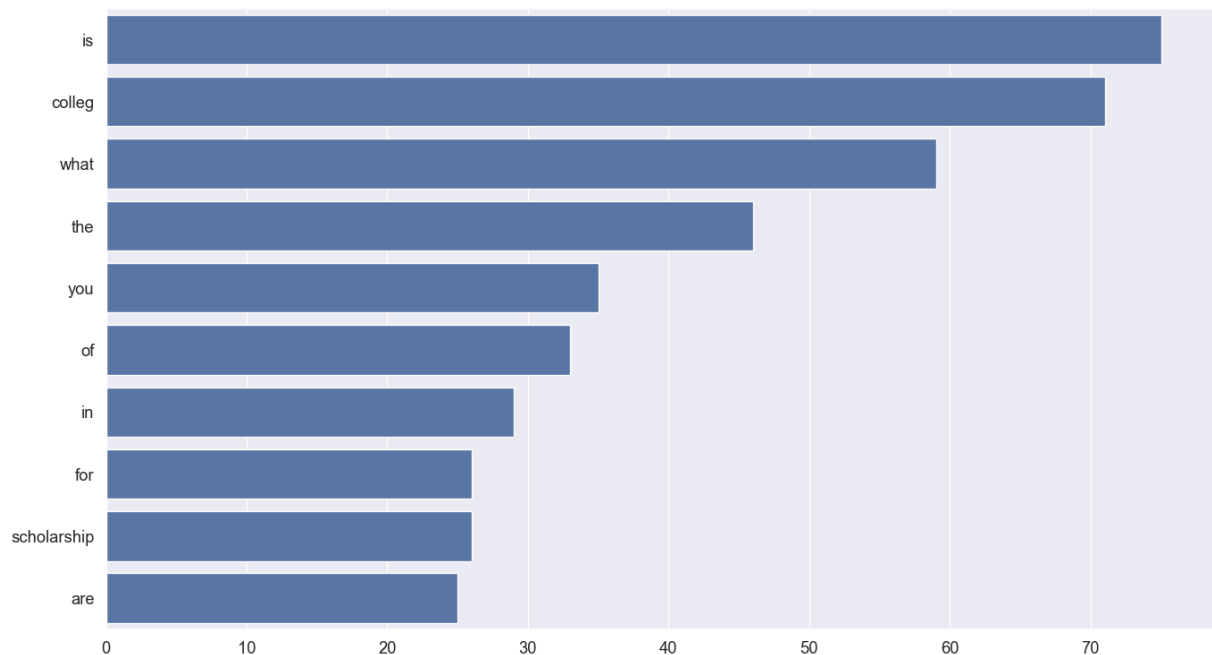
```
In [30]: from collections import Counter  
        counter = Counter(corpus)  
        most_common = counter.most_common(10)  
        most_common = dict(most_common)  
        most_common
```



```
Out[30]: {'is': 75,  
          'colleg': 71,  
          'what': 59,  
          'the': 46,  
          'you': 35,  
          'of': 33,  
          'in': 29,  
          'for': 26,  
          'scholarship': 26,  
          'are': 25}
```

```
In [31]: def get_top_text_ngrams(corpus, n,g):  
          vec = CountVectorizer(ngram_range=(1, 1)).fit(corpus)  
          bag_of_words = vec.transform(corpus)  
          sum_words = bag_of_words.sum(axis=0)  
          words_freq = [(word, sum_words[0, idx]) for word, idx in vec.vocabulary_.items(  
          words_freq =sorted(words_freq, key = lambda x: x[1], reverse=True)  
          return words_freq[:n]
```

```
In [32]: plt.figure(figsize = (16,9))  
          most_common_uni = get_top_text_ngrams(df.Pattern,10,1)  
          most_common_uni = dict(most_common_uni)  
          sns.barplot(x=list(most_common_uni.values()),y=list(most_common_uni.keys()));
```



```
In [33]: df2.head()
```

```
Out[33]:
```

	Pattern	Tag
0	Hi	greeting
1	How are you?	greeting
2	Is anyone there?	greeting
3	Hello	greeting
4	Good day	greeting

```
In [34]: labels = df2['Tag'].unique().tolist()
labels = [s.strip() for s in labels]
labels
```

```
Out[34]: ['greeting',
'goodbye',
'creator',
'name',
'hours',
'number',
'course',
'fees',
'location',
'hostel',
'event',
'document',
'floors',
'syllabus',
'library',
'infrastructure',
'canteen',
'menu',
'placement',
'ithod',
'computerhod',
'extchod',
'principal',
'sem',
'admission',
'scholarship',
'facilities',
'college intake',
'uniform',
'committee',
'random',
'swear',
'vacation',
'sports',
'salutaion',
'task',
'ragging',
'hod']
```

```
In [35]: num_labels = len(labels)
id2label = {id:label for id, label in enumerate(labels)}
label2id = {label:id for id, label in enumerate(labels)}
id2label
```

```
Out[35]: {0: 'greeting',
1: 'goodbye',
2: 'creator',
3: 'name',
4: 'hours',
5: 'number',
6: 'course',
7: 'fees',
8: 'location',
9: 'hostel',
10: 'event',
11: 'document',
12: 'floors',
13: 'syllabus',
14: 'library',
15: 'infrastructure',
16: 'canteen',
17: 'menu',
18: 'placement',
19: 'ithod',
20: 'computerhod',
21: 'extchod',
22: 'principal',
23: 'sem',
24: 'admission',
25: 'scholarship',
26: 'facilities',
27: 'college intake',
28: 'uniform',
29: 'committee',
30: 'random',
31: 'swear',
32: 'vacation',
33: 'sports',
34: 'salutaion',
35: 'task',
36: 'ragging',
37: 'hod'}
```

```
In [36]: label2id
```

```
Out[36]: {'greeting': 0,
          'goodbye': 1,
          'creator': 2,
          'name': 3,
          'hours': 4,
          'number': 5,
          'course': 6,
          'fees': 7,
          'location': 8,
          'hostel': 9,
          'event': 10,
          'document': 11,
          'floors': 12,
          'syllabus': 13,
          'library': 14,
          'infrastructure': 15,
          'canteen': 16,
          'menu': 17,
          'placement': 18,
          'ithod': 19,
          'computerhod': 20,
          'extchod': 21,
          'principal': 22,
          'sem': 23,
          'admission': 24,
          'scholarship': 25,
          'facilities': 26,
          'college intake': 27,
          'uniform': 28,
          'committee': 29,
          'random': 30,
          'swear': 31,
          'vacation': 32,
          'sports': 33,
          'salutaion': 34,
          'task': 35,
          'ragging': 36,
          'hod': 37}
```

```
In [37]: df2['labels'] = df2['Tag'].map(lambda x: label2id[x.strip()])
df2.head()
```

```
Out[37]:
```

	Pattern	Tag	labels
0	Hi	greeting	0
1	How are you?	greeting	0
2	Is anyone there?	greeting	0
3	Hello	greeting	0
4	Good day	greeting	0

```
In [38]: X = list(df2['Pattern'])
X[:5]
```

```
Out[38]: ['Hi', 'How are you?', 'Is anyone there?', 'Hello', 'Good day']
```

```
In [39]: y = list(df2['labels'])
y[:5]
```

```
Out[39]: [0, 0, 0, 0, 0]
```

```
In [40]: X_train,X_test,y_train,y_test = train_test_split(X,y,random_state = 123)
```

```
In [41]: model_name = "bert-base-uncased"
max_len = 256

tokenizer = BertTokenizer.from_pretrained(model_name,
                                         max_length=max_len)

model = BertForSequenceClassification.from_pretrained(model_name,
                                                    num_labels=num_labels,
                                                    id2label=id2label,
                                                    label2id = label2id)
```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
In [42]: train_encoding = tokenizer(X_train, truncation=True, padding=True)
test_encoding = tokenizer(X_test, truncation=True, padding=True)
```

```
In [43]: full_data = tokenizer(X, truncation=True, padding=True)
```

```
In [44]: class DataLoader(Dataset):

    def __init__(self, encodings, labels):

        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):

        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):

        return len(self.labels)
```

```
In [45]: train_dataloader = DataLoader(train_encoding, y_train)
test_dataloader = DataLoader(test_encoding, y_test)
```

```
In [46]: fullDataLoader = DataLoader(full_data, y_test)
```

```
In [47]: def compute_metrics(pred):

    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='micro')
    acc = accuracy_score(labels, preds)

    return {
        'Accuracy': acc,
        'F1': f1,
        'Precision': precision,
        'Recall': recall
    }
```

```
In [48]: training_args = TrainingArguments(
    output_dir='./output',
    do_train=True,
    do_eval=True,
    num_train_epochs=100,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=16,
    warmup_steps=100,
    weight_decay=0.05,
    logging_strategy='steps',
    logging_dir='./multi-class-logs',
    logging_steps=50,
    eval_strategy="steps",
    eval_steps=50,
    save_strategy="steps",
    load_best_model_at_end=True
)
```

```
In [49]: trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataloader,
    eval_dataset=test_dataloader,
    compute_metrics= compute_metrics
)
```

```
In [50]: trainer.train()
```

[1000/1000 01:33, Epoch 100/100]

Step	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
50	3.496900	3.243073	0.107843	0.041868	0.054182	0.088172
100	2.655700	2.143876	0.637255	0.512950	0.569432	0.550000
150	1.136500	0.942696	0.862745	0.780049	0.802525	0.797980
200	0.344600	0.556436	0.921569	0.927994	0.943011	0.937634
250	0.122300	0.468277	0.911765	0.911788	0.920968	0.926882
300	0.060200	0.452181	0.931373	0.929391	0.943011	0.943011
350	0.041800	0.466724	0.921569	0.921850	0.934946	0.937634
400	0.028300	0.463279	0.921569	0.921850	0.934946	0.937634
450	0.025900	0.464782	0.921569	0.921850	0.934946	0.937634
500	0.023700	0.463298	0.931373	0.929391	0.943011	0.943011
550	0.019800	0.463737	0.931373	0.929391	0.943011	0.943011
600	0.017600	0.469490	0.931373	0.929391	0.943011	0.943011
650	0.015500	0.475064	0.921569	0.921850	0.934946	0.937634
700	0.015300	0.471171	0.931373	0.929391	0.943011	0.943011
750	0.014500	0.472043	0.931373	0.929391	0.943011	0.943011
800	0.013200	0.476097	0.931373	0.929391	0.943011	0.943011
850	0.012800	0.477752	0.931373	0.929391	0.943011	0.943011
900	0.012900	0.478089	0.931373	0.929391	0.943011	0.943011
950	0.012800	0.479436	0.931373	0.929391	0.943011	0.943011
1000	0.012300	0.479828	0.931373	0.929391	0.943011	0.943011

```
Out[50]: TrainOutput(global_step=1000, training_loss=0.4041301252245903, metrics={'train_runtime': 94.1967, 'train_samples_per_second': 321.667, 'train_steps_per_second': 10.616, 'total_flos': 264789670748400.0, 'train_loss': 0.4041301252245903, 'epoch': 100.0})
```

```
In [51]: q=[trainer.evaluate(eval_dataset=df2) for df2 in [train_dataloader, test_dataloader]]
pd.DataFrame(q, index=["train", "test"]).iloc[:, :5]
```

[19/19 00:00]

```
Out[51]:
```

	eval_loss	eval_Accuracy	eval_F1	eval_Precision	eval_Recall
train	0.010107	0.996700	0.998509	0.998747	0.998355
test	0.479828	0.931373	0.929391	0.943011	0.943011

```
In [52]: def predict(text):

    inputs = tokenizer(text, padding=True, truncation=True, max_length=512, return_tensors='pt')
    outputs = model(**inputs)

    probs = outputs[0].softmax(1)
    pred_label_idx = probs.argmax()
    pred_label = model.config.id2label[pred_label_idx.item()]

    return probs, pred_label_idx, pred_label
```

```
In [53]: text = "Hello"
predict(text)
```

```
Out[53]: (tensor([[9.9376e-01, 2.9152e-04, 1.8633e-04, 6.3930e-04, 1.4596e-04, 4.0864e-04,
8.8468e-05, 9.7566e-05, 1.0214e-04, 1.0850e-04, 2.4759e-04, 8.2971e-05,
2.0951e-04, 9.1758e-05, 7.5741e-05, 1.9913e-04, 7.3006e-05, 1.8177e-04,
4.7549e-05, 9.6713e-05, 2.0444e-04, 4.4762e-05, 1.2031e-04, 2.1198e-04,
3.9131e-05, 7.2070e-05, 4.7395e-05, 8.2099e-05, 2.2951e-04, 1.3374e-04,
7.2437e-05, 1.4203e-04, 1.0003e-04, 1.6741e-04, 4.7384e-04, 5.1833e-04,
8.5684e-05, 1.1697e-04]], device='cuda:0', grad_fn=<SoftmaxBackward0>),
tensor(0, device='cuda:0'),
'greeting')
```

```
In [54]: model_path = "chatbot"
trainer.save_model(model_path)
tokenizer.save_pretrained(model_path)
```

```
Out[54]: ('chatbot\\tokenizer_config.json',
'chatbot\\special_tokens_map.json',
'chatbot\\vocab.txt',
'chatbot\\added_tokens.json')
```

```
In [55]: pip install xformers
```


Requirement already satisfied: xformers in c:\users\badd\miniconda3\lib\site-packages (0.0.33.post2)
Requirement already satisfied: numpy in c:\users\badd\miniconda3\lib\site-packages (from xformers) (2.2.4)
Requirement already satisfied: torch==2.9.1 in c:\users\badd\miniconda3\lib\site-packages (from xformers) (2.9.1+cu126)
Requirement already satisfied: filelock in c:\users\badd\miniconda3\lib\site-packages (from torch==2.9.1->xformers) (3.20.2)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\badd\miniconda3\lib\site-packages (from torch==2.9.1->xformers) (4.15.0)
Requirement already satisfied: sympy>=1.13.3 in c:\users\badd\miniconda3\lib\site-packages (from torch==2.9.1->xformers) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in c:\users\badd\miniconda3\lib\site-packages (from torch==2.9.1->xformers) (3.6.1)
Requirement already satisfied: Jinja2 in c:\users\badd\miniconda3\lib\site-packages (from torch==2.9.1->xformers) (3.1.3)
Requirement already satisfied: fsspec>=0.8.5 in c:\users\badd\miniconda3\lib\site-packages (from torch==2.9.1->xformers) (2025.12.0)
Requirement already satisfied: setuptools in c:\users\badd\miniconda3\lib\site-packages (from torch==2.9.1->xformers) (73.0.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\badd\miniconda3\lib\site-packages (from sympy>=1.13.3->torch==2.9.1->xformers) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\badd\miniconda3\lib\site-packages (from Jinja2->torch==2.9.1->xformers) (2.1.5)
Note: you may need to restart the kernel to use updated packages.

In [56]: `model_path = "chatbot"`

```
model = BertForSequenceClassification.from_pretrained(model_path)
tokenizer = BertTokenizerFast.from_pretrained(model_path)
chatbot = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)
```

Device set to use cuda:0

In [57]: `chatbot("Hello")`

Out[57]: `[{'label': 'greeting', 'score': 0.9937639832496643}]`

In [58]: `def chat(chatbot):`

```
    print("Chatbot: Hi! I am your virtual assistance, Feel free to ask, and I'll do my best to help you. Type 'quit' to exit the chat\n\n")

    text = input("User: ").strip().lower()

    while(text != 'quit'):

        score = chatbot(text)[0]['score']

        if score < 0.8:
            print("Chatbot: Sorry I can't answer that\n\n")
            text = input("User: ").strip().lower()
            continue

        label = label2id[chatbot(text)[0]['label']]
```

```
response = random.choice(intents['intents'][label]['responses'])  
  
print(f"Chatbot: {response}\n\n")  
  
text = input("User: ").strip().lower()
```

In [59]: `chat(chatbot)`

Chatbot: Hi! I am your virtual assistance, Feel free to ask, and I'll do my best to provide you with answers and assistance..
Type 'quit' to exit the chat

Chatbot: I can answer to low-intermediate questions regarding college

Chatbot: Sorry I can't answer that

Chatbot: College is open 8am-5pm Monday-Saturday!

Chatbot: [here](ADD YOU GOOGLE MAP LINK HERE)

Chatbot: Sorry I can't answer that